Advanced Programming Project
Digital Media Library

Name: Atharva Rakshe

Student ID: 100003165

Course: Advanced Programming

Instructor: Esteban Pozo

Date: 12.12.2025

Abstract

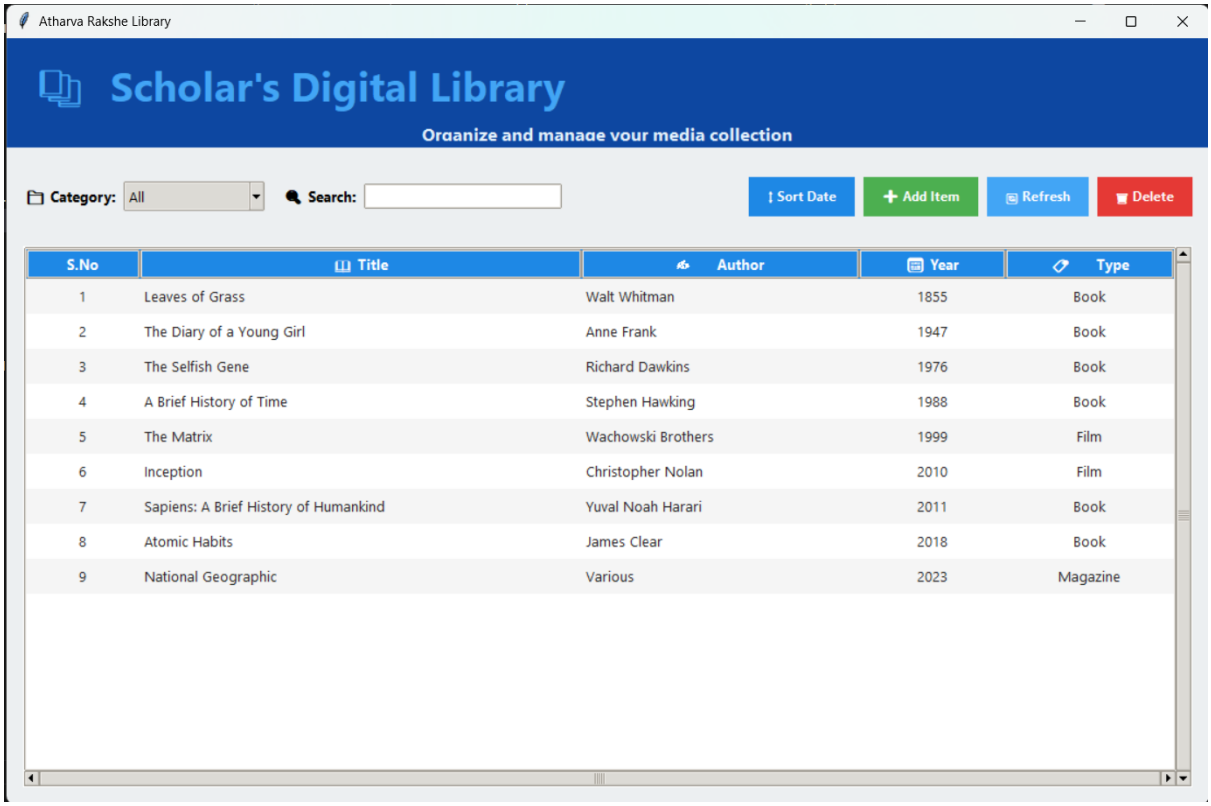
The Digital Media Library is a Python-based application built to help users easily organize and manage different types of media, such as books, films, and magazines. It brings together a Tkinter graphical interface, a Flask backend server, and JSON storage to provide full CRUD functionality with persistent data handling.

Through a user-friendly interface, people can add, edit, delete, search, filter, and sort media records. The system communicates with the backend using RESTful API requests, and it includes fallback logic, so the application keeps working even if the server is temporarily unavailable.

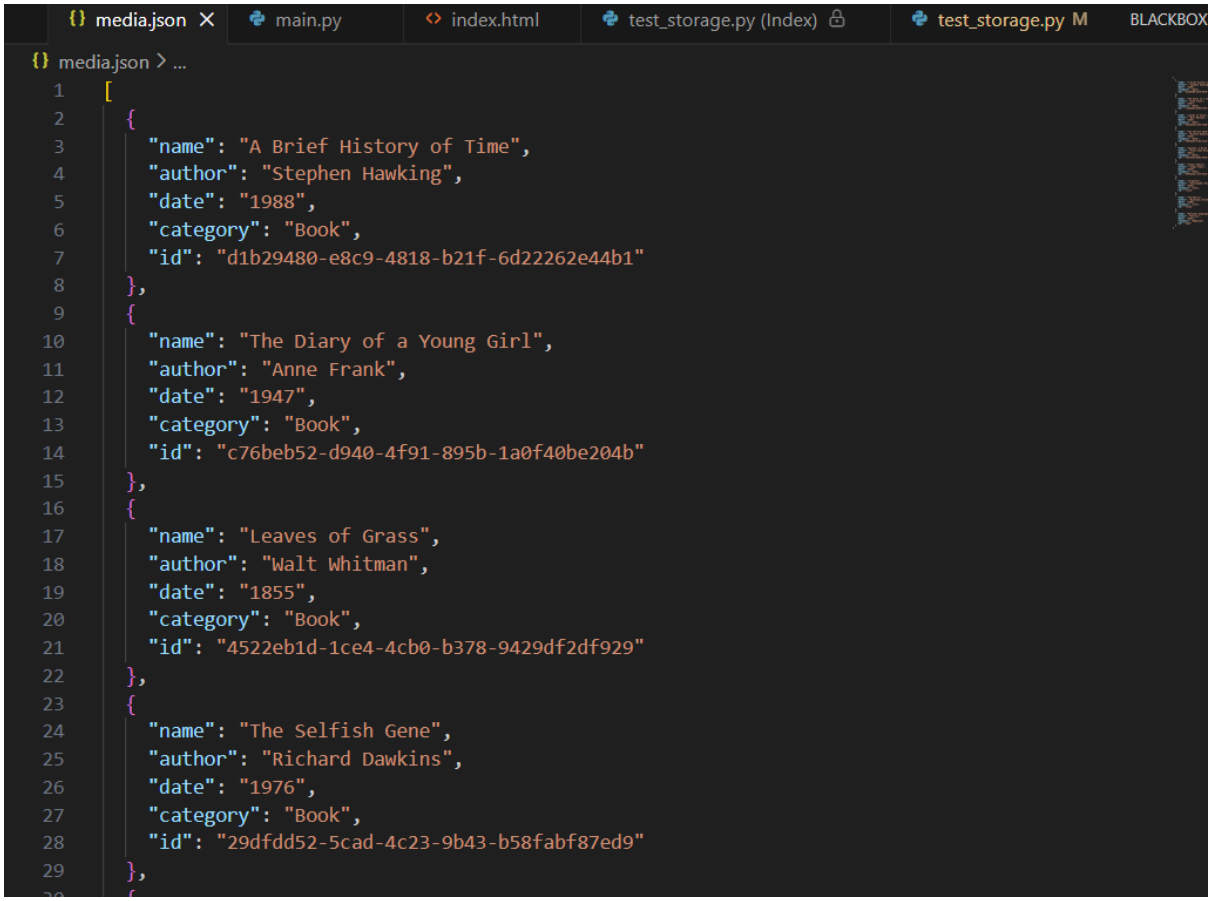
This project highlights modular software design, structured error handling, and smooth integration between the GUI and backend components. The report walks through the core ideas behind the system, how it was built, the testing methods used, challenges encountered along the way, improvements made, and an overall evaluation of the system's performance and potential for future expansion.

Introduction

The Digital Media Library is a Python-based application designed to organize and manage books, films, and magazines. Developed for the Advanced Programming course, the system combines a Tkinter graphical interface, a Flask backend, and JSON storage to provide users with a simple and efficient way to add, edit, delete, search, and filter media items. This report explains the project concept, implementation, testing, and key improvements made during development.



Picture 1: Online Library Interface



Picture 2: JSON File

Task 1 – Concept and Requirements:

Functional Requirements:

- Display all media items stored in the backend
- Display media by category (Book, Film, Magazine)
- Search media using **exact-match** name lookup
- Display full metadata for any selected item
- Create new media items
- Delete existing media items
- Update media entries (in your implementation this is enabled through edit forms)
- Persist all data to JSON so the app restores the library on startup
- Backend must expose HTTP endpoints for:
    - Listing all items
    - Listing items by category
    - Searching by exact name

- Retrieving metadata of a specific item
- Creating items
- Deleting items

Non-Functional Requirements:

- Usability: GUI must be intuitive, with clear buttons, input fields, and tables
- Maintainability: Modular separation into main.py, server.py, storage.py
- Reliability: Fallback logic if backend is unreachable (your program implements this).
- Consistency: Use of UUIDs for unique identification.
- Persistence: JSON used as long-term storage.
- Error Handling: Detect invalid input, missing fields, corrupted files, network errors.

Architecture Overview

Frontend (Tkinter):

- Loads and displays media
- Sends requests to backend
- Allows CRUD operations through GUI forms
- Uses Treeview for formatted display
- Includes dropdown filtering, exact search, and metadata display

Backend (Flask API):

- /media — list all media
- /media?category= — list media by category
- /media/search — exact-match search
- /media/<id> — retrieve, update, delete
- /media POST — create new items

Storage Layer (JSON):

- Media saved as list of dictionaries
- Each entry includes name, author, date, category, ID
- Ensures safe read/write using lock
- Automatically generates UUIDs

GUI Design Sketch (Text Description)

- A header/title
- A Treeview table showing all media

- Dropdown menu for category filter
- Search bar for exact-match lookup
- Buttons: Add Media, Edit, Delete, Sort by Date
- Pop-up windows for creating/editing items
- A details panel for metadata display

Communication Between Frontend and Backend

- Frontend uses Python requests module to call the Flask backend
- Backend returns JSON responses
- If backend fails, frontend switches to local JSON fallback mode
- CRUD operations are mirrored in both modes

Anticipated Exceptions

- Backend offline → use fallback storage
- Invalid year input → GUI validation error
- Missing fields → backend rejects request
- File corruption → storage recreates JSON file
- Duplicate IDs → prevented via UUID generator
- Network timeout → caught in try/except

Task 2 – Implementation

Frontend Implementation (main.py)

- Built completely with Tkinter
- Uses Treeview widget to show structured media
- Loads data from backend or JSON, depending on availability
- Implements Add, Delete, Edit through popup forms
- Events trigger API calls or local updates
- Sorting and filtering included
- Abstraction through functions: load_media(), create_media(), delete_media(), etc.

Backend Implementation (server.py)

- GET /media
- GET /media?category=
- GET /media/search?name=
- POST /media
- DELETE /media/<id>
- PUT /media/<id>

Backend Uses:

- Input validation
- JSON response formatting
- Delegation to storage layer

Storage Implementation (storage.py)

- Load JSON file
- Save updated list
- Create, update, delete items
- Assign UUIDs
- Guarantee consistent format

Task 3 – Testing

- Backend Test — validate item creation through API
- Backend Test — exact-match search endpoint
- Backend Test — deletion route functionality
- Frontend Test — save/load JSON fallback
- Manual GUI Testing — interaction flow and error handling

  - Backend tests used Python's unittest or manual endpoint testing
  - Verified correctness of status codes and JSON responses
  - Frontend fallback tested by shutting down backend
  - GUI behavior validated with different input cases

Task 4 – Discussion & Reflection

Weaknesses

- JSON is not ideal for large datasets
- GUI freezes during long backend operations
- No asynchronous operations
- No authentication or multi-user features

Strengths

- Clean architecture
- Modular and maintainable
- Robust fallback mode
- UUID-based consistency
- Intuitive GUI for end-users

Future Improvements

- Replace JSON with SQLite or PostgreSQL
- Add asynchronous backend calls
- Enhance search (partial matching, filters)
- Add themes and UI improvements
- Introducing error logs and audit trails

Real-world operational considerations

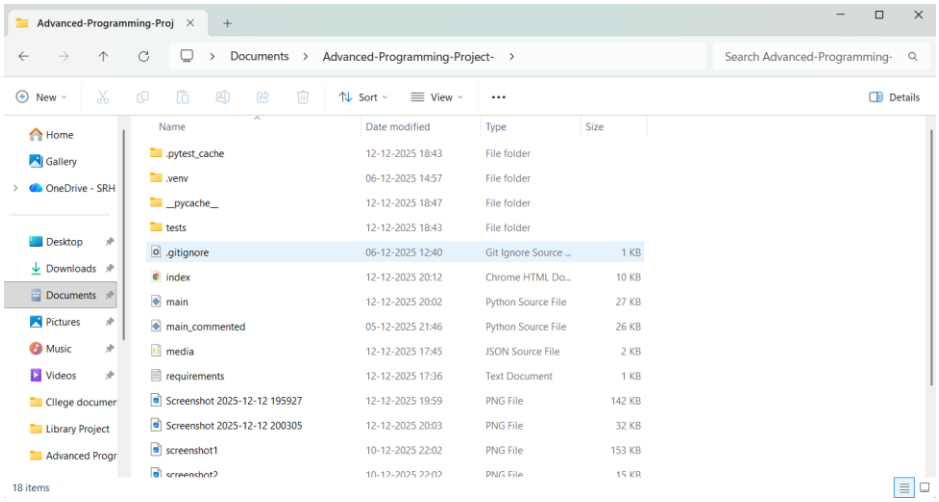
- Need for secure login if used by multiple users
- Need for automated backups
- Need for deployed backend (cloud or local server)

References

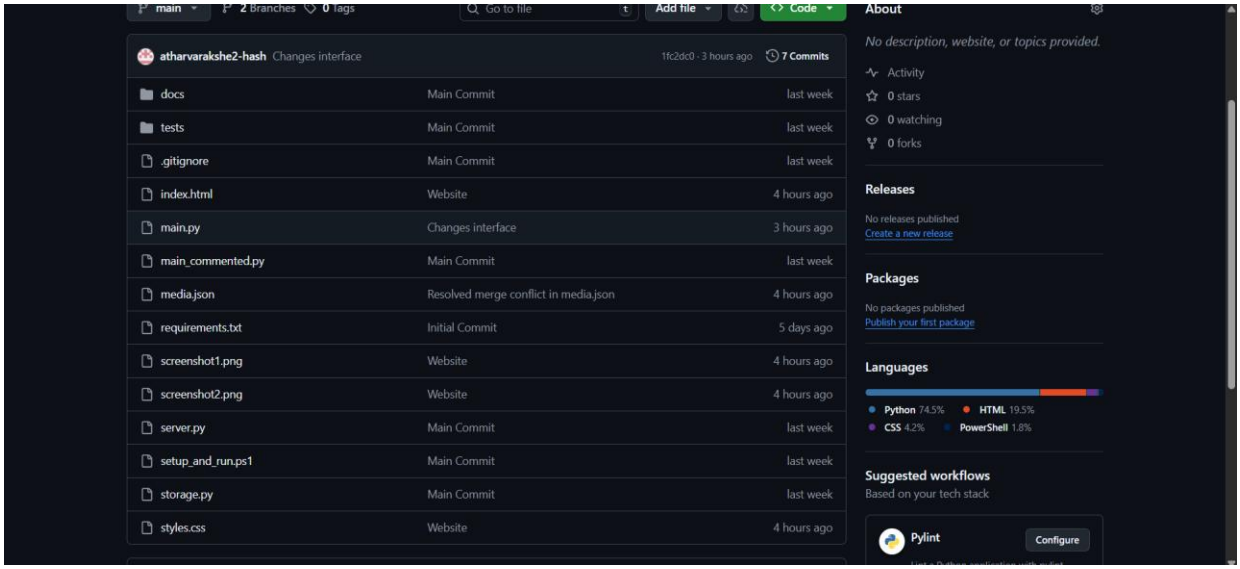
- Python Official Documentation - https://docs.python.org/3/
- Tkinter Documentation - https://docs.python.org/3/library/tkinter.html
- Flask Documentation - https://flask.palletsprojects.com/

- Requests Library - https://requests.readthedocs.io/
- pytest Documentation - https://docs.pytest.org/
- JSON Standard - RFC 8159, The JavaScript Object Notation (JSON) Data Interchange Forma
- UUID Standard - RFC 4122, A Universally Unique Identifier (UUID) URN Namespace
- Software Design Patterns - Gang of Four (GoF) Design Patterns
- Clean Code Principles - Robert C. Martin, "Clean Code: A Handbook of Agile Software Craftsmanship
- GitHub Copilot and ChatGPT

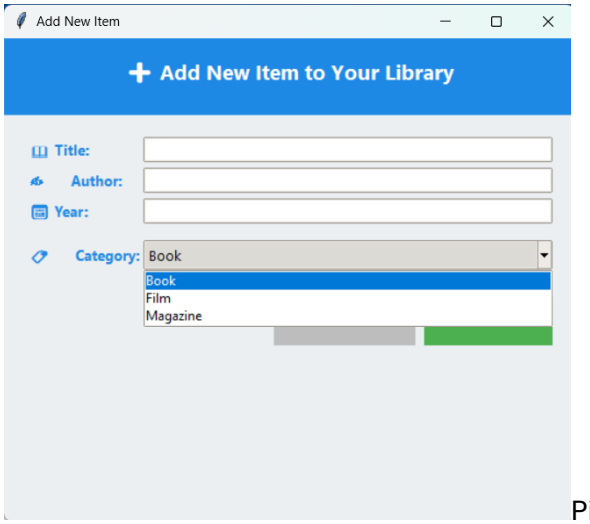
Appendix



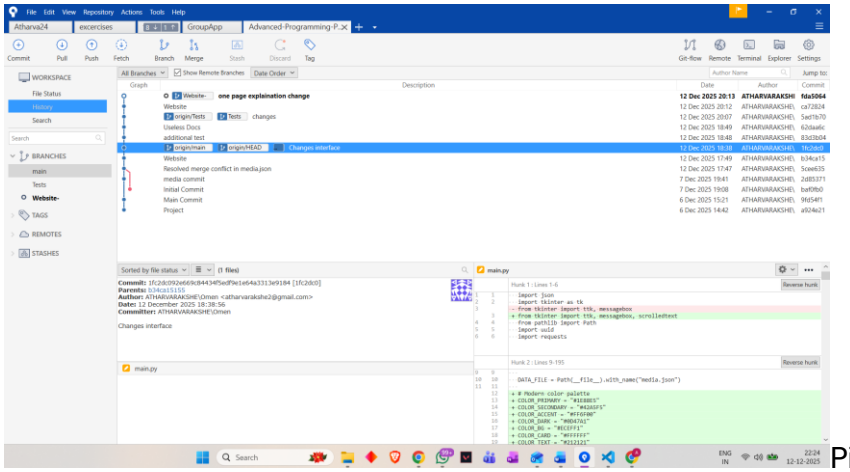
Picture 3: Supporting Folder



Picture 4: GitHub



Picture 5: New Item Adding Interface



Picture 6: Sourcetree

The appendix provides supporting technical documentation for the Digital Media Library project. The system consists of three core Python modules: *main.py*, which implements the Tkinter-based frontend responsible for displaying media items, handling user interactions, and managing operations such as creating, editing, deleting, sorting, and searching records through an intuitive interface; *server.py*, which provides the Flask backend offering RESTful API endpoints used to retrieve, create, update, and delete media items, validate input, and return structured JSON responses; and *storage.py*, which manages persistent JSON file operations using thread-safe read and write mechanisms while assigning unique UUID identifiers to each media entry. The project fulfills all assignment requirements, including displaying all media items, categorizing items by type, performing exact-match searches, and supporting full CRUD functionality. Backend fallback ensures reliability when the server is unavailable. Testing included validation of backend CRUD operations, exact-match search behavior, and fallback storage functionality, along with manual GUI testing to ensure error messages, input validation, and interface responsiveness. The appendix also includes screenshots of the project folder structure and the interface used to add new items to the library, which are sufficient representations of the implementation. Additionally, the requirement-mapping overview confirms alignment with assignment specifications, and AI usage is disclosed: parts of the written documentation were developed with assistance from AI language tools, while all code, design decisions, and testing were carried out independently by me.

AI Prompts Used:

- "I am developing a Python project for my university assignment. I need to build an online library system using Flask as the backend and Tkinter as the frontend. Can you help me design the architecture and explain what modules I should create?"

- "Also, for the table of the list of books add some decent cool looking color which will differentiate each row."

- "Compare the PDF with the Python file I have shared and see whether all the requirements in the pdf are met by my python file and also let me know what all shall I change or rectify to get an A."

THANK YOU